

```
<style>
body {
font-family: 'Arial', sans-serif;
font-size: 11pt;
}
h1, h2, h3, h4, h5, h6 {
font-family: 'Arial', sans-serif;
font-size: 14pt;
}
}</style>
```

Guía de Tests E2E (Playwright) para Desarrolladores

¡Hola, equipo de desarrollo! 🤖

QA ha preparado esta guía rápida y súper sencilla para que entiendan de qué van los "Tests Automatizados" (E2E) que hemos dejado en la carpeta `tests/e2e/3er_Sprint/`.

No se preocupen, no es magia oscura, es bastante simple.

1. ¿Qué diablos es un Test E2E? 🤖

E2E significa **End-to-End** (De principio a fin).

Imaginen que tenemos a una personita invisible (un robot) navegando por la aplicación exactamente como lo haría un usuario real:

1. Abre el navegador.
2. Hace clic en un botón.
3. Escribe texto en los inputs.
4. Y mira si en la pantalla aparece lo que debería aparecer.

¿Para qué sirve?

Para evitar que cuando arreglen algo, rompan otra cosa sin darse cuenta. Si el robot hace clic en "Guardar" y la página se cae, el test fallará y nos avisará antes de mandarlo a producción.

2. Detalle y Justificación de cada Test Case (Sprint 3) 📁

Para que sepan exactamente qué evalúa cada bloque de código y por qué se consideró importante automatizarlo, aquí tienen la explicación de todos los casos de prueba de este sprint:

🔑 HU-11: Vista de Administrador (HU_11.spec.js)

- **TC-85 (Métricas reales):** Verifica que las tarjetas principales muestren números reales (≥ 1). *Justificación:* Asegura que el dashboard sirva de algo y no muestre ceros o errores de base de datos.
- **TC-86 (Métrica de documentos):** Verifica la tarjeta de documentos subidos. *Justificación:* QA identificó que este valor está "hardcodeado" (fijo en 4,302), el test avisa si sigue así para que lo hagan dinámico.
- **TC-87 (Tabla Usuarios Recientes):** Confirma que la tabla de últimos usuarios tiene encabezados y filas. *Justificación:* El admin necesita ver quién se registra; si la tabla no renderiza, el admin queda ciego.
- **TC-88A (Toggle Estado Visual):** Verifica que al hacer clic en el botón de estado, el texto cambie a Inactivo/Activo. *Justificación:* Validación visual de estado. El guardado en BD se prueba manual hasta que el endpoint esté listo.
- **TC-90 (Panel de Actividad):** Verifica que el timeline derecho muestre acciones. *Justificación:* Asegura que los logs de sistema (Activity Service) estén llegando al front.
- **TC-91 (Navegación Calendario):** Verifica que se pueda cambiar de mes en el calendario. *Justificación:* Evita que un script roto bloquee la navegación de fechas.
- **TC-92 (Sidebar navegación):** Da clic en todos los links laterales. *Justificación:* Si los links del sidebar se rompen, el panel entero queda inútil.
- **TC-93 (Filtro por estado):** Busca usuarios filtrando por "activos". *Justificación:* Asegura que los endpoints de filtrado AJAX o DOM funcionen bien.
- **TC-94 (Vista Portafolios Stats):** Verifica las estadísticas exclusivas de portafolios en el admin. *Justificación:* Los administradores evalúan métricas de uso reales.
- **TC-95 y TC-96 (Seguridad Accesos):** Intenta entrar a `/admin` sin sesión o con usuario normal. *Justificación:* **Crítico.** Evita que cualquier usuario se ponga a borrar o ver métricas sensibles.
- **TC-97 (Responsive móvil):** Abre el panel simulando un iPhone y prueba el menú de hamburguesa. *Justificación:* Muchos administradores entran desde su celular; el menú lateral oculto debe funcionar.
- **TC-98 (Cerrar menú flotante):** Abre el menú de acciones de usuario y hace clic fuera para ver si se cierra. *Justificación:* Detalles de UX que hacen la aplicación menos frustrante.
- **TC-99 (Logout Admin):** Abre el modal de cerrar sesión y confirma. *Justificación:* Garantizar que la sesión se destruya correctamente.
- **TC-151 (Avatar visible):** Revisa que la foto de perfil superior derecha aparezca. *Justificación:* Comprueba que la UI base carga correctamente tras un inicio de sesión.

📄 HU-12: Reportes y CV (HU_12.spec.js)

- **TC-100 (Acceso a Reportes):** Intenta cargar la vista. *Justificación:* Prueba básica (Smoke Test) de que la ruta `/reportes` no tira error 500.
- **TC-101 (Cambio de Plantilla):** Hace clic en otro botón de plantilla y verifica que la UI se actualice. *Justificación:* Asegura que el selector de diseños funcione con JavaScript interactivo.
- **TC-102 (Datos Mapeados):** Busca que haya texto de más de 50 caracteres en el CV visualizado. *Justificación:* El CV generado no puede estar vacío, debe inyectar la información real del perfil de base de datos.
- **TC-103A (Foto de Perfil):** Revisa si la etiqueta `<img>` del CV se carga. *Justificación:* Una URL rota en el CV arruina la exportación PDF.
- **TC-104 (Botón Exportar):** Revisa que el botón de PDF exista. *Justificación:* Sin botón de exportar, todo el módulo de reportes no tiene sentido para el usuario final.
- **TC-106 (Acceso denegado CV):** Prueba abrir un reporte sin haber iniciado sesión. *Justificación:* Protege la API de generación contra bots anónimos.
- **TC-107 (Aislamiento de CV):** Extrae el texto del CV y verifica que contenga el email del usuario logueado. *Justificación:* **Privacidad.** Evita el grave error de que a un usuario le salga el currículum de otra persona.
- **TC-108 (Responsive CV):** Comprime la pantalla y revisa que el CV siga viéndose bien. *Justificación:* La gente ama su CV desde el celular.
- **TC-152 (Título UI):** Revisa el título `<h2>`. *Justificación:* Validación rápida del renderizado de `layout.app`.

📁 HU-13: Portafolios (HU_13.spec.js)

- **TC-109 y 109B (Crear y Listar):** Manda datos válidos vía POST al backend y espera respuesta de éxito, luego revisa que aparezca listado. *Justificación:* Es la funcionalidad principal del módulo de portafolios (CRUD).
- **TC-109C (Portafolios UI Dashboard):** Va al home del usuario y verifica que ahí se listen las tarjetas. *Justificación:* Confirma la integración entre la vista Dashboard y los datos creados en Evidencias.
- **TC-110, TC-111, TC-112 (Validaciones):** Intenta crear proyectos vacíos o con URLs que no son URLs. *Justificación:* El backend debe rebotar peticiones malformadas siempre para mantener la base de datos limpia.
- **TC-116 (Editar):** Toma el id del proyecto creado y le actualiza el nombre. *Justificación:* Verificar el método PUT/POST de actualización.
- **TC-118 y TC-119 (Seguridad Edición Ajena):** Intenta modificar un portafolio con ID 99999 (que no es suyo). *Justificación:* **Seguridad severa.** Un atacante interceptando requests no debe poder alterar o eliminar portafolios ajenos.
- **TC-120 (Límite de caracteres):** Intenta mandar un nombre de proyecto con más de 255 caracteres. *Justificación:* Previene desbordamiento y errores SQL al truncar strings largos.
- **TC-153 (Tabs visuales):** Comprueba que la botonera superior exista. *Justificación:* Validación de UI básica.

HU-15: Notificaciones (HU_15.spec.js)

- **TC-121 (Alertar a Todos):** El Admin manda una alerta global. *Justificación:* Confirmar que el broadcast masivo no crashee si hay muchos usuarios.
- **TC-122 y TC-124A (Alertas Individuales):** Selecciona y manda a un usuario, y prueba mandar a nadie. *Justificación:* Las validaciones de relaciones en la BD deben existir.
- **TC-123 y TC-125 (Mensajes sin título o excesivos):** Envía formularios vacíos o de mil caracteres. *Justificación:* Filtra el ruido e inconsistencias desde el frontend.
- **TC-127 (Recepción Destino):** El usuario destino inicia sesión y verifica su bandeja de entrada. *Justificación:* Asegura el canal de comunicación completo (End-to-End real).
- **TC-128 (Marcar Leída):** El usuario da clic para que desaparezca el icono de "nuevo". *Justificación:* Verificar que la BD actualiza la columna leída.
- **TC-129 (Seguridad Notif):** Un usuario normal trata de hacer un POST al controlador de admin para enviar una alerta. *Justificación:* Garantiza la existencia de middlewares como `es_admin`.
- **TC-131A (Historial Admin):** El admin pide la lista completa para ver lo que ha enviado. *Justificación:* Auditoría de sistema.
- **TC-132 (Validación enum):** Intenta enviar un tipo falso de notificación. *Justificación:* Seguridad contra manipulación de forms en el navegador.
- **TC-154 (Icono Campana):** Chequea la UI superior derecha. *Justificación:* Asegurar navegación rápida.

HU-16: Idiomas (HU_16.spec.js)

- **TC-133, 134, 135 (Cambios de Idioma):** Navega por `/lang/en`, `/lang/es`, etc. y lee si los menús cambiaron. *Justificación:* El switch básico de locales de Laravel.
- **TC-136 (Persistencia Session):** Recarga la página después de cambiar de idioma. *Justificación:* Asegura que el valor se guardó en memoria y no desaparece al primer F5.
- **TC-137 y TC-138 (Idioma inválido y Path Traversal):** Prueba `/lang/de` y `/lang/../../../../etc/passwd`. *Justificación:* **Seguridad crítica.** Si el backend hace includes dinámicos con esa variable, alguien podría inyectar archivos maliciosos.
- **TC-139 (Redirect Loop):** Evalúa a dónde te lleva `/lang/en`. *Justificación:* Detecta bucles de redirección donde la página se queda recargando eternamente.
- **TC-140 (Conservar Sesión):** Cambia el idioma y trata de entrar al menú de nuevo. *Justificación:* Laravel a veces pierde las cookies de sesión al hacer ciertas redirecciones de middleware; esto verifica que sigas logueado.
- **TC-141 (Textos Perfil):** Comprueba la traducción en sub-módulos como el de perfil. *Justificación:* Integración transversal de diccionarios de traducción.
- **TC-142 y TC-155 (Default español y Dropdown):** Comprueba UI base del idioma. *Justificación:* La app asume el idioma por defecto configurado.

Requisitos de Cliente - Soft Delete (HU_CLIENTE_CRUD.spec.js)

- **TC-147 y TC-148 (CRUD Habilidades):** El bot crea una Habilidad y le cambia el nivel. *Justificación:* Para tener un 100% de confianza en que un usuario normal sí puede llenar su trayectoria.
- **TC-149 y TC-150 (Soft Delete Cliente):** El bot elimina la habilidad que acaba de crear y verifica que ya no aparece en su lista. *Justificación (Atención Devs):* El cliente requería que esto **solo lo ocultara al usuario** (Soft Delete) pero no de la BD. El test automatizado pasará (porque desaparece visualmente), pero QA ya levantó un ticket indicando que backend está haciendo un Hard Delete físico `->delete()`.

3. ¿Cómo leer un error si un Test falla?

Si ejecutan los tests y sale texto ROJO, **no entren en pánico**. Aquí un manual de supervivencia:

Error común 1: "Timeout exceed" o "locator no encontrado"

```
Error: locator.click: Timeout 30000ms exceeded.
Call log:
  - waiting for locator('button#btn-guardar')
```

¿Qué significa en español?

El robot se cansó de esperar a que apareciera un botón con el id `btn-guardar`.

¿Cómo lo arreglo?

Probablemente ustedes le cambiaron el `id` o la `class` al botón en el blade. Revisen que el test busque las mismas clases que dejaron en el código de producción.

Error común 2: "Expected X to be Y"

```
Expected: 200
Received: 500
```

¿Qué significa en español?

El test esperaba recibir un código de éxito de la API (200), pero el servidor devolvió un error interno de Laravel (500).

¿Cómo lo arreglo?

Significa que el backend se rompió por dentro (quizás falta una columna en su base de datos o un error de sintaxis PHP). Abran su archivo `storage/logs/laravel.log` para ver qué explotó.

4. ¿Cómo corro estos tests en mi máquina local?

Es sumamente fácil. Abran su terminal en la raíz del proyecto y escriban:

```
npm playwright test
```

Si quieren ver cómo el navegador se abre y el bot hace los clics "en vivo y a todo color" (muy útil para entender visualmente qué está pasando), usen:

```
npx playwright test --ui
```

¡Eso es todo! Si los tests pasan, todo está bien. Si fallan, arreglen el código o avisen a QA. 🚀